

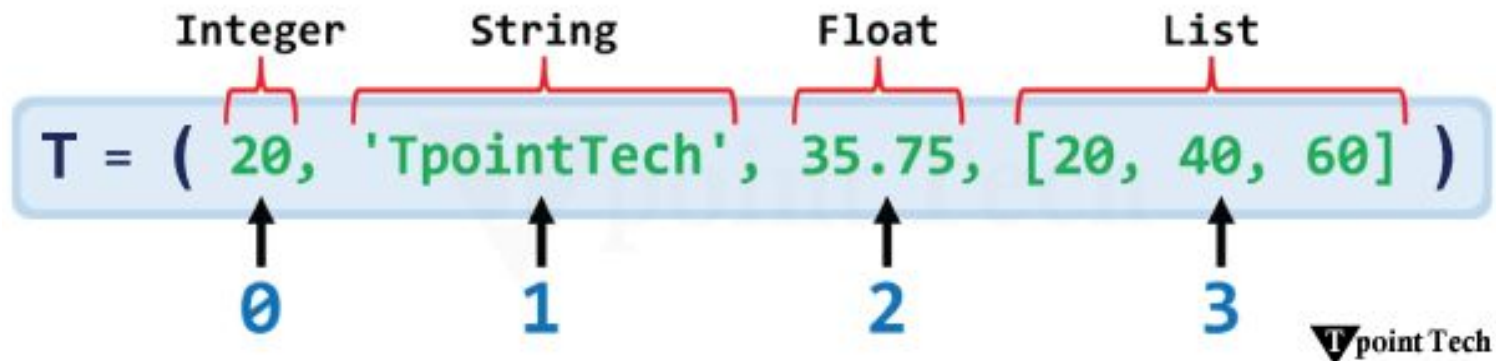
2. TUPLES IN PYTHON

TUPLES

Built-in data structure in Python for storing a collection of objects.

Characteristics of Python Tuples

- ❑ **Ordered:** Each data element in a tuple has a particular order that never changes.
- ❑ **Immutable:** The data elements of tuple cannot be changed once initialized.
- ❑ **Allow duplicate elements.**



DEFINING TUPLES

- ❑ `T = (20, 'Sreeraj', 35.75, [20, 40, 60])`
- ❑ `T1 = ( )` #Empty Tuple
- ❑ `T2 = ( 5, )` # Tuple having only one element
- ❑ `nested_tuple = ('String', {4: 5, 6: 2, 8: 2}, (5, 3, 15, 6))`

ACCESSING TUPLE ELEMENTS

- ❑ `T[0] = 20` (Integer)
- ❑ `T[1] = ,Sreeraj'` (String)
- ❑ `T[2] = 35.75` (Float)
- ❑ `T[3] = [20, 40, 60]` (Nested List)

ADD / UPDATE ELEMENTS TO A TUPLE

1. Add element by concatenation

```
t = (10, 20, 30)
t = t + (40,)
print(t) # (10, 20, 30, 40)
```

3. Adding multiple elements

```
t = (10, 20)
t = t + (30, 40, 50)

print(t)
# (10, 20, 30, 40, 50)
```

2. Convert tuple → list, add element → convert back

```
t = (1, 2, 3)
temp = list(t)
temp.append(4)
t = tuple(temp)
```

```
print(t)
# (1, 2, 3, 4)
```

ADD / UPDATE ELEMENTS TO A TUPLE

4. Adding elements using unpacking

```
t = (10, 20)
```

```
t = (*t, 30)
```

```
print(t)
```

```
# (10, 20, 30)
```

✗ Not allowed

```
t = (10, 20)
```

```
t.append(30) # Error
```

```
t[0] = 99    # Error
```

Deleting a Tuple

Syntax: del tuple_name

Example:

- ❑ sampleTuple = ("Apple", "Mango", "Banana", "Orange", "Guava", "Berries")
- ❑ del sampleTuple

SLICING IN TUPLE

Slicing a tuple means dividing a tuple into smaller tuples with the help of indexing

□ **Syntax:**

□ tuple[start:end: Step]

Example:

- `t = (10, 20, 30, 40, 50, 60)`
- `print(t[1:4])` # (20, 30, 40)
- `print(t[:3])` # (10, 20, 30)
- `print(t[3:])` # (40, 50, 60)
- `print(t[0:6:2])` # (10, 30, 50)
- `print(t[-4:-1])` # (30, 40, 50)
- `print(t[::-1])` # (60, 50, 40, 30, 20, 10)

LOOPING TUPLES

```
# creating a tuple
fruits_tuple = ("mango", "orange", "banana", "apple", "papaya")
print("Looping with 'for' Loop:")
i = 1
for fruit in fruits_tuple:
    print(i, "-", fruit)
    i += 1

print("\nLooping with 'while' Loop:")
j = 0
while j < len(fruits_tuple):
    print(j + 1, "-", fruits_tuple[j])
    j += 1
```


TUPLE OPERATIONS IN PYTHON

S. No.	Python Operator	Python Expression	Result	Description
1	+	(2, 4, 6) + (3, 5, 7)	(2, 4, 6, 3, 5, 7)	Concatenation
2	*	('Apple',) * 3	('Apple', 'Apple', 'Apple')	Repetition
3	in, not in	4 in (2, 4, 6, 8, 10)	True	Membership

TUPLE METHODS

1. The count() Method

Syntax: tuple_name.count(item)

Example:

```
T1 = (0, 2, 3, 6, 4, 2, 5, 6, 3, 2, 2, 6, 7, 2, 7, 8, 0, 1, 9, 1)
```

```
resultOne = T1.count(2)
```

2.The index() Method

Syntax: tuple_name.index(item,start,stop)

```
T1 = (0, 2, 3, 6, 4, 2, 5, 6, 3, 2, 2, 6, 7, 2, 7, 8, 0, 1, 9, 1)
```

```
resultOne = T1.index(2)
```

```
resultTwo = T1.index(2, 5)
```

DIFFERENCE B/W LIST AND TUPLE

Feature	List	Tuple
Mutability	We can modify a list by adding or removing items (Mutable).	We cannot modify a tuple by adding or removing items (Immutable).
Performance	Lists are slower due to mutability.	Tuples are faster due to their static size and immutability.
Memory Usage	List uses more memory.	Tuple uses less memory.
Methods	Python List offers more built-in methods. (e.g., append, extend, remove)	Python Tuple offers fewer built-in methods. (e.g., index, count)
Syntax	We can define a list using square brackets [].	We can define a tuple using parentheses ().
Iteration Speed	Iteration is slightly slower in Lists due to their dynamic nature.	Iteration is faster in Tuples as they are immutable.
Storage Efficiency	Lists require extra memory for dynamic allocation.	Tuples are more memory-efficient
Error Safety	Lists are prone to accidental changes.	Tuples provide data integrity to prevent errors.
Use Case	Lists are used when data needs to change.	Tuples are used when data should remain constant.